

Task 1:string similarity

```
import java.io.*;
import java.util.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

class Result {

    /**
     * Complete the 'stringSimilarity' function below.
     *
     * The function is expected to return a LONG.
     * The function accepts STRING s as parameter.
     */

    public static long stringSimilarity(String s) {

        int n = s.length();

        int[] z = new int[n];

        int left = 0;
        int right = 0;

        for (int i = 1; i < n; i++) {

            if (i <= right) {
                z[i] = Math.min(right - i + 1, z[i - left]);
            }

            while (i + z[i] < n &&
                s.charAt(z[i]) == s.charAt(i + z[i])) {
                z[i]++;
            }

            if (i + z[i] - 1 > right) {
                left = i;
                right = i + z[i] - 1;
            }
        }

        return z[0] + z[1] + ... + z[n-1];
    }
}
```

```

        right = i + z[i] - 1;
    }
}

long similarity = n;

for (int i = 1; i < n; i++) {
    similarity += z[i];
}

return similarity;
}
}

public class Solution {

    public static void main(String[] args) throws IOException {

        BufferedReader bufferedReader =
            new BufferedReader(
                new InputStreamReader(System.in)
            );

        BufferedWriter bufferedWriter =
            new BufferedWriter(
                new FileWriter(System.getenv("OUTPUT_PATH"))
            );

        int t = Integer.parseInt(
            bufferedReader.readLine().trim()
        );

        IntStream.range(0, t).forEach(tItr -> {

            try {

                String s = bufferedReader.readLine();

                long result = Result.stringSimilarity(s);

```

```

        bufferedWriter.write(
            String.valueOf(result)
        );

        bufferedWriter.newLine();

    } catch (IOException ex) {
        throw new RuntimeException(ex);
    }
});

bufferedReader.close();
bufferedWriter.close();
}
}

```

Output:

input (stdin)

2

ababaa

aa

Your Output (stdout)

11

3

Expected Output

11

3

Task 2: Repeated substring pattern

```

class Solution {
    public boolean repeatedSubstringPattern(String s) {
        int n = s.length();

        for (int len = 1; len <= n / 2; len++) {
            if (n % len != 0) {
                continue;
            }

            String pattern = s.substring(0, len);

```

```

        boolean valid = true;

        for (int i = len; i < n; i += len) {
            if (!s.substring(i, i + len).equals(pattern)) {
                valid = false;
                break;
            }
        }

        if (valid) {
            return true;
        }

        return false;
    }
}

```

Output:

Input

s =
"abab"

Output

true

Expected

true

Task 3:two strings

```

import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

class Result {

```

```

/*
 * Complete the 'twoStrings' function below.
 *
 * The function is expected to return a STRING.
 * The function accepts following parameters:
 * 1. STRING s1
 * 2. STRING s2
 */

public static String twoStrings(String s1, String s2) {
    // Write your code here

    boolean[] seen = new boolean[26];

    for (char c : s1.toCharArray()) {
        seen[c - 'a'] = true;
    }

    for (char c : s2.toCharArray()) {
        if (seen[c - 'a']) {
            return "YES";
        }
    }

    return "NO";
}

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new
FileWriter(System.getenv("OUTPUT_PATH")));
    }
}

```

```

        int q = Integer.parseInt(bufferedReader.readLine().trim());

        IntStream.range(0, q).forEach(qItr -> {
            try {
                String s1 = bufferedReader.readLine();

                String s2 = bufferedReader.readLine();

                String result = Result.twoStrings(s1, s2);

                bufferedWriter.write(result);
                bufferedWriter.newLine();
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        });

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

Output:

Input (stdin)

2
hello
world
hi
world

Your Output (stdout)

YES
NO

Expected Output

YES
NO

Task 4:rotate string

```

class Solution {
    public boolean rotateString(String s, String goal) {

```

```

        if (s.length() != goal.length()) {
            return false;
        }

        return (s + s).contains(goal);
    }
}

```

Output:

Input

```

s =
"m"
goal =
"f"

```

Output

false

Expected

false

Task 5:Mars exploration

```

import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

class Result {

    /**
     * Complete the 'marsExploration' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts STRING s as parameter.
     */
}

```

```

    public static int marsExploration(String s) {
        // Write your code here

        int count = 0;

        String message = "SOS";

        for (int i = 0; i < s.length(); i++) {
            if (s.charAt(i) != message.charAt(i % 3)) {
                count++;
            }
        }

        return count;
    }

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new
FileWriter(System.getenv("OUTPUT_PATH")));

        String s = bufferedReader.readLine();

        int result = Result.marsExploration(s);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

Output:

Input (stdin)

SOSSPSSQSSOR

Your Output (stdout)

3

Expected Output

3

Task 6:find all anagrams in a string

```
class Solution {
    public List<Integer> findAnagrams(String s, String p) {
        List<Integer> result = new ArrayList<>();

        if (s.length() < p.length()) {
            return result;
        }

        int[] pCount = new int[26];
        int[] windowCount = new int[26];

        // Count characters in p
        for (char c : p.toCharArray()) {
            pCount[c - 'a']++;
        }

        int windowSize = p.length();

        // Build the first window
        for (int i = 0; i < windowSize; i++) {
            windowCount[s.charAt(i) - 'a']++;
        }

        // Check first window
        if (Arrays.equals(pCount, windowCount)) {
            result.add(0);
        }

        // Slide the window
        for (int i = windowSize; i < s.length(); i++) {
            // Add new character
            windowCount[s.charAt(i) - 'a']++;

            // Remove leftmost character
            windowCount[s.charAt(i - windowSize) - 'a']--;

            // Check if current window is an anagram
            if (Arrays.equals(pCount, windowCount)) {
```

```

        result.add(i - windowSize + 1);
    }
}

return result;
}
}

```

Output:

Input

```

s =
"cbaebabacd"
p =
"abc"

```

Output

```
[0,6]
```

Expected

```
[0,6]
```

Task 7:palindrome index

```

import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

class Result {

    /**
     * Complete the 'palindromeIndex' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts STRING s as parameter.
     */
}

```

```

public static int palindromeIndex(String s) {
    // Write your code here

    int left = 0;
    int right = s.length() - 1;

    while (left < right) {
        if (s.charAt(left) != s.charAt(right)) {

            // Try removing the left character
            if (isPalindrome(s, left + 1, right)) {
                return left;
            }

            // Try removing the right character
            if (isPalindrome(s, left, right - 1)) {
                return right;
            }

            return -1;
        }

        left++;
        right--;
    }

    // Already a palindrome
    return -1;
}

private static boolean isPalindrome(String s, int left, int right) {
    while (left < right) {
        if (s.charAt(left) != s.charAt(right)) {
            return false;
        }

        left++;
        right--;
    }

```

```

    }

    return true;
}

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new
FileWriter(System.getenv("OUTPUT_PATH")));

        int q = Integer.parseInt(bufferedReader.readLine().trim());

        IntStream.range(0, q).forEach(qItr -> {
            try {
                String s = bufferedReader.readLine();

                int result = Result.palindromeIndex(s);

                bufferedWriter.write(String.valueOf(result));
                bufferedWriter.newLine();
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        });

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

Output:

Compiler Message

Success

Input (stdin)

3
aaab
baa
aaa

Your Output (stdout)

3
0
-1

Expected Output

3
0
-1

Task 8: find the index of the first occurrence

```
class Solution {
    public int strStr(String haystack, String needle) {
        int n = haystack.length();
        int m = needle.length();

        if (m > n) {
            return -1;
        }

        for (int i = 0; i <= n - m; i++) {
            int j = 0;

            while (j < m && haystack.charAt(i + j) == needle.charAt(j)) {
                j++;
            }

            if (j == m) {
                return i;
            }
        }

        return -1;
    }
}
```

Output:

Input

haystack =
"sadbutsad"
needle =
"sad"

Output

0

Expected

0

Task 9:longest palindromic string

```
class Solution {
    public String longestPalindrome(String s) {
        if (s.length() < 2) {
            return s;
        }

        int start = 0;
        int end = 0;

        for (int i = 0; i < s.length(); i++) {

            // Odd-length palindrome
            int len1 = expandFromCenter(s, i, i);

            // Even-length palindrome
            int len2 = expandFromCenter(s, i, i + 1);

            int len = Math.max(len1, len2);

            if (len > end - start + 1) {
                start = i - (len - 1) / 2;
                end = i + len / 2;
            }
        }

        return s.substring(start, end + 1);
    }

    private int expandFromCenter(String s, int left, int right) {
        while (left >= 0 &&
            right < s.length() &&
            s.charAt(left) == s.charAt(right)) {

            left--;
            right++;
        }

        return right - left - 1;
    }
}
```

Output:

Input

s =

"babad"

Output

"bab"

Expected

"bab"

Task 10:circular palindrome

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

class Result {

    /**
     * Complete the 'circularPalindromes' function below.
     *
     * The function is expected to return an INTEGER_ARRAY.
     * The function accepts STRING s as parameter.
     */

    public static List<Integer> circularPalindromes(String s) {
        // Write your code here

        int n = s.length();

        // Duplicate the string to handle circular rotations.
        String doubled = s + s;
```

```

/*
 * Transform:
 * "abc" -> "#a#b#c#"
 *
 * This lets Manacher handle both odd and even
 * length palindromes with the same algorithm.
 */
int m = 2 * doubled.length() + 1;
char[] transformed = new char[m];

for (int i = 0; i < m; i++) {
    if (i % 2 == 0) {
        transformed[i] = '#';
    } else {
        transformed[i] = doubled.charAt(i / 2);
    }
}

// Manacher radius array.
int[] radius = manacher(transformed);

/*
 * Segment tree for range maximum queries.
 *
 * For every rotation we need:
 * "What is the largest palindrome radius
 * among these possible centers?"
 */
SegmentTree tree = new SegmentTree(radius);

List<Integer> result = new ArrayList<>();

for (int start = 0; start < n; start++) {

    // First and last character positions
    // in the transformed string.
    int left = 2 * start + 1;
    int right = 2 * (start + n - 1) + 1;

```

```

    int low = 1;
    int high = n;
    int best = 1;

    /*
     * Binary search for the longest palindrome.
     */
    while (low <= high) {
        int len = (low + high) / 2;

        /*
         * A palindrome of length len must have
         * its center between:
         *
         * left + len - 1
         * right - len + 1
         */
        int queryLeft = left + len - 1;
        int queryRight = right - len + 1;

        if (queryLeft <= queryRight &&
            tree.query(queryLeft, queryRight) >= len) {

            best = len;
            low = len + 1;

        } else {
            high = len - 1;
        }
    }

    result.add(best);
}

return result;
}

/*

```

```

    * Manacher's algorithm.
    *
    * radius[i] = maximum number of transformed
    * characters we can expand around i.
    */
private static int[] manacher(char[] s) {
    int n = s.length;
    int[] radius = new int[n];

    int center = 0;
    int right = 0;

    for (int i = 0; i < n; i++) {

        if (i < right) {
            radius[i] = Math.min(
                right - i,
                radius[2 * center - i]
            );
        }

        while (i - radius[i] - 1 >= 0 &&
            i + radius[i] + 1 < n &&
            s[i - radius[i] - 1] ==
            s[i + radius[i] + 1]) {

            radius[i]++;
        }

        if (i + radius[i] > right) {
            center = i;
            right = i + radius[i];
        }
    }

    return radius;
}

/*

```

```

* Segment tree supporting range maximum query.
*/
private static class SegmentTree {

    private int[] tree;
    private int size;

    SegmentTree(int[] values) {
        size = 1;

        while (size < values.length) {
            size *= 2;
        }

        tree = new int[2 * size];

        for (int i = 0; i < values.length; i++) {
            tree[size + i] = values[i];
        }

        for (int i = size - 1; i > 0; i--) {
            tree[i] = Math.max(tree[2 * i], tree[2 * i + 1]);
        }
    }

    int query(int left, int right) {
        int result = 0;

        left += size;
        right += size;

        while (left <= right) {

            if ((left & 1) == 1) {
                result = Math.max(result, tree[left]);
                left++;
            }

            if ((right & 1) == 0) {

```

```

        result = Math.max(result, tree[right]);
        right--;
    }

    left /= 2;
    right /= 2;
}

return result;
}
}

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new
FileWriter(System.getenv("OUTPUT_PATH")));

        int n = Integer.parseInt(bufferedReader.readLine().trim());

        String s = bufferedReader.readLine();

        List<Integer> result = Result.circularPalindromes(s);

        bufferedWriter.write(
            result.stream()
                .map(Object::toString)
                .collect(joining("\n"))
            + "\n"
        );

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

```
}  
}
```

Output:

Input (stdin)

13

aaaaabbbbbaaaa

Your Output (stdout)

12

12

10

8

8

9

11

13

11

9

8

8

10

Expected Output

12

12

10

8

8

9

11

13

11

9

8

8

10